

Q.1 main()
 { int i=40;
 do
 { pf("%d", i++);
 } while (5, 4, 3, 2, 1, i-41);
 return 0;
 }

O/P = 40

Q.2 main()
 { int a=10;
 if (pf("%d", a>=10)-10)
 for(;;)
 break;
 else;
 }

O/P = 1

Q.3 vector printing times

main()
 { int x;
 for(x=-1, x<=10; x++)
 { if(x<5)
 continue;
 else
 break;
 pf("vector");
 }
 return 0;
 }

0 times
vector
printed

NO O/P

Q.4 minimum no. of iteration in do while 1

Q.5 auto int a=5
 main()

{

 }

Compile time
error.

auto can
be global

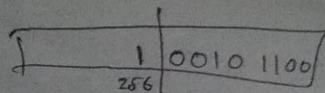
Q.11 main()

int i=300

char *ptr=&i;

*++ptr=2

printf("x.d", i);



O/P = 556

Q.12

main()

int a[2][3][2] = { { { 2, 2, 3 }, { 7, 8, 3 }, { 3, 4, 3 } }, { { 2, 2, 3 }, { 2, 3, 3 }, { 3, 4, 3 } } }

printf("x.d", **(*arr+1)+2+7);

O/P = 16

Q.13

main()

char thought[2][30] = { "Don't walk in front of me", "I am not follow" };

printf("x.c x.c", *(thought[0]+9), *(*(thought[1]+5)));

O/P = K-

Q.14

main()

int i=257;

int *iptr=&i;

printf("x.d %d", *((char*)iptr), *((char*)iptr+2));

O/P is 11

Q.15

main()

static int a[] = { 0, 1, 2, 3, 4 };

int *p[] = { a, a+1, a+2, a+3, a+4 };

int **ptr = p;

ptr+4;

```

    pf("%d %d %d", ptr-P, *ptr-a, **ptr);
    *++ptr;
    pf("%d %d %d", ptr-P, *ptr-a, **ptr);
}

```

O/P ⇒ 1 1 2 2 2

(16) main()

```

    void *vp;
    char ch='g', *cp="goofy";
    int j=20;
    vp=&ch;
    pf("%c", *(char*)vp);
    vp=&j;
    pf("%d", *(int*)vp);
    vp=&cp;
    pf("%s", (char*)vp+3);
}

```

O/P ⇒ g2ofy

(17) main()

```

    char a[5][5], flag;
    a[0][0]='A';
    flag = (a==*a) && (*a==a[0]);
    pf("%d", flag);
}

```

O/P ⇒ 1

(18) main()

```

    func();
}
func(int i)

```

O/P ⇒ two

```

    static char str[] = {"one", "two", "three", "four"};
    pf("%s", str[i++]);
}

```

68 271

20) main()
 { int argc, i;
 i = argc - 1;
 char *argv[3] = {"abcde", "fghij", "klmno"};
 while (++argv, --argc, --i);
 pf("%s", *argv);
}

```

(21) main()
{
    union a;
    int i;
    char c[2];
    i;
    union a v;
    v.i = 256;
    printf("%d %d %d", v.i, v.c[0]++, v.c[1]++);
}

```

```

22 main ( )
{
    enum bool { true, false };
    if ( true == (2 == 3) )
        pf ("vector");
    else pf ("Institute");
}

```

O/P
⇒ institute

```

23 struct st
{
    int x;
    int y;
};
main()
{
    struct st *p = {10, 20};
    printf("%d %d", p->x, p->y);
}

```

O/P $\Rightarrow$ Segmentation fault.

```

24 main()
{
    union
    {
        struct {
            char c[2];
            char ch[2];
        } s;
        struct {
            short int i;
            short int j;
        } st;
    } U = {12, 1, 15, 1};
    printf("%d %d", U.st.i, U.st.j);
}

```

O/P $\Rightarrow$ 268 271

```

20 main()
{
    #define big(a,b) a>b?a:b
    #define swap(a,b) temp=a; a=b; b=temp;
    main()
    {
        int a=3, b=5, temp=0;
        if (3+big(a,b)) > b swap(a,b);
        printf("%d %d", a, b);
    }
}

```

O/P $\Rightarrow$ 50

architecture
file time
res

```

31) main()
    #define int char;
    main()
    {
        int i=65;
        printf("%d", size of(i));
    }

```

OP = 1

```

32) main() #define float ptr float f1, f2 & p1 &
    #define int ptr int &
    float fptr f1, f2;
    int ptr p1, p2;

```

OP ⇒ float, int

```

33) char *gc()
    {
        static char x[1024];
        return x;
    }

```

O/P ⇒ Second string

```

main()
{
    char *g1 = "first string";
    strcpy(gc, g1);
    g1 = gc;
    strcpy(g1, "second string");
    printf("%s", gc);
}

```

```

34) main()
    {
        printf("%d", --2);
    }

```

O/P ⇒ Compile time error.

35

Sun
S
U
n
sun
sun

enum mode { green, red, orange, blue, white }
 main()
 {
 green = green + 1
 printf("%d %d", green, r)
 }

Compile time error

45 union { int x;
 char y;
 struct { char x;
 char y;
 int z;
 } p;
 } q;

size of q is 8 bytes

46 main()
 {
 int arr[] = {10, 20, 30, 40, 50};
 char *p;
 p = (char *)arr;
 }

How to print 50

⇒ *(p+16)

or
 printf("%d", *((int*)p)+4)

C paper

① main()

```

{
    int x=123;
    i = printf("c"+"t");
    for (x=0; x<=i; x++)
    {
        pf("%d", x);
    }
    return 0;
}
    
```

C++0123

②

```

for (i=0; i<5; i++)
    for (j=i; j>0; j--)
        x=i+j+1;
pf("x-d", x);
    
```

4+1+1=6

③

```

fun(int i) {
    if (i%2)
        return 0;
    else
        return 1;
}
    
```

```

main() {
    i=3;
    p=fun(i);
    q=fun(i);
    pf("x-d", p);
}
    
```

i=1

④ scope of a global variable which is declare as static within a file.

⑤ void fun()

```

{
    int x=0;
    static int y=0;
    x++, y++;
    pf("x-d->y-d", x, y);
}
    
```

```

main {
    fun();
    fun();
}
    
```

1 → 1
 1 → 2

```

⑤ main()
{
    int a = 5;
    {
        int b = 10;
        ++b;
        ++a;
        int a = 20;
        ++a;
        a = ++b;
        ++a;
        ++b;
        printf("%d %d", a, b);
    }
    printf("%d", a);
}

```

7, 13, 7

- ⑦ what is use of static functions → static int sum
 security purpose
 ↓
 not access local file
- ⑧ Even if integer / float arguments are supplied at command prompt → ① integer
 ② double
 ③ string
- ⑨ return type of calloc → void *
- ⑩ malloc → stdlib.h
- ⑪ char *p = (char *)0;
 int *q = (int *)0;
 float *r = (float *)0;
 double *s = 0;
 a = p + 1, b = q + 1, c = r + 1, d = s + 1

Ans ⇒ 1, 4, 4, 8

- ⑫ realloc → void * realloc (void *, size)
 free → void free (void *)

- ⑬ func(int a, int b)
 { return (a = (a == b)); }

main()

{ int process(); func(); }

printf("the value is %d", process(func, 3, 6));

Ans = 0

```

Process (int (*Print) (c), int Val1, int Val2);
{
    return ((*Pf) (Val1, Val2));
}

```

Ans $\Rightarrow$ 0 Call back function

14

```

char *g();
{
    static char x[1024];
    return x;
}

main()
{
    char *g1 = "first string";
    strcpy(g(), g1);
    g1 = g();
    strcpy(g, "second string");
    Pf("%s", g());
}

```

Ans $\Rightarrow$ Second String

15

```

main()
{
    int i, argc;
    i = argc = 3;
    char *arg[3] = {"abcde", "fghi", "klmno"};
    char **argv = arg;
    while (++argv, --argc, --i)
        Pf("%s", *argv);
}

```

Ans $\Rightarrow$ fghi klmno.

16

```

main()
{
    union a
    {
        int i;
        char C[2];
    };
    union a u;
    u.i = 256;
    Pf("%d %d %d", u.i,
        u.C[0]++,
        u.C[1]++);
}

```

Ans $\Rightarrow$ 513, 0, 1

16) enum mode { green, red, orange, blue, white }
green = green + 1

Ans → Compile-time error.

17) enum { false, true }
main() {
int i = 1;
do { printf("%d", i);
i++;
if (i < 15)
continue;
} while (false);
}

Ans = 1

18) if ne or more nb → nested structure.

20) main() {
enum bool { True, false };
if (True == (2 == 3))
printf("vector");
else
printf("institute --");
}

Ans → ...vector

21) main() {
struct xx {
int x = 3;
char name[] = "hello";
};

error

we can't initialize inside structure.

23) #define a 10
main() {
printf("%d ---", a);
foo();
printf("%d", a);
}
void foo() {
#undef a;
#define a 50
}

Ans

10 10

24) #define big(a,b) a > b ? a : b;
#define swap(a,b) {
temp = a;
a = b;
b = temp;
}
void main() {
int a = 3, b = 5, temp = 0;
if ((3 + big(a,b)) > b swap(a,b))
printf("%d %d", a, b);
}

Ans → 5, 0

lenovo

(25) A default setting enumerator start from 0

(25) #define Cal (a,b) (a+b)/a-b

void main()

{ int a=20, b=10;

Pf ("x.d", Cal (a+4, b-2));

Ans → 4

(26) #define MAN(x,y) ((x)>(y))?(x):(y);

main()

{ int i=10, j=5, k=0;

k=MAN(++i, j++);

Pf ("x.d x.d x.d", i, j, k);

return 0;

Ans → 12 6 12

(27) #define f(g1,g2) g1##g2

main()

{ int var12=100;

Pf ("x.d", f(var, 12));

}

O/P → 100

(29) Source text ⇒ "Be my friend\n".

main()

{ FILE *fs, *ft;

char c[10];

fs = fopen("source.txt", "r");

c[0] = getc(fs);

fseek(fs, 0, SEEK_END);

fseek(fs, -3L, SEEK_CUR);

fgetc(c, 5, fs);

put(c);

return 0;

Ans → nd

(30) The fopen() return 0

(31) main()

{ FILE *fp;

Pf ("x.d", "++EOF");

}

O/P → Error

++(-1)

can't

```

32) main (int argc, char *argv[])
{
    if (argc == 1)
        printf("error");
        exit(1);
    while (*argv, --argc)
    {
        printf("%s", *++argv);
        while (argc, *argv)
            printf("%c\n", *(++argv));
        putchar("\n");
    }
}

```

i/p ⇒ Sun Run

O/p ⇒
 Sun
 n
 Run
 n
 n

33) mention ~~at least~~ at least 3 facilities provided by a preprocessor.

- Include header files
- Replace macro
- Conditional compilation

34) va-list is predefined datatype in stdarg.h.

```

35) main()
{
    int i = -1;
    +i;
    printf("v.d v.d", i, +i);
}

```

Ans → -1, -1

36) ternary operator.

$x > y ? x : y$

37) size of ptr1 & ptr2?

struct x { int; char; }

Ans 4, 4

```
38 main()
{ int x=10;
  b x 5
  pf("%d", x);
}
```

Ans $\Rightarrow$ 10

39 diff b/w
#include < > & #include "
↑ predefined ↑ user defined

40 unimary operator

Size of ++, ~, !, --, &

```
41 main()
{ static char a[] = {"black", "white", "yellow", "violet"};
  char *p[] = {a+3, a+2, a+1, a};
  p = p+3;
  ++*p;
  pf("%s", --*++p+3);
}
```

Ans $\Rightarrow$ te

```
42 main()
{ float a=123.4, b=456.8;
  a^=b^=a^=b;
  pf("%d %d", a, b);
}
```

Ans $\Rightarrow$ error

bitwise can operate on float.

```
43 main()
{ int i;
  for (i=0; i<5; i++)
  { int j=0;
    pf("%d", i);
    i++;
  }
}
```

Ans $\Rightarrow$ 10 10101010

```
44 main()
{ int n=10;
  int func(int);
  pf("%d", func(n));
}

int func(int n)
{ if (n>0)
  return (n+func(n-2));
  else return 0;
}
```

Ans $\Rightarrow$ 30

① → double-8 byte Paper 2 / const

~~long~~ → main is not a keyword

Paper 2

④ break should be used in loops & switch cases.
→ compile time error

⑤ #include <stdio.h>
main()
{ int const x=0;
switch (5/4/3)
{ case x:
case x+1;
case x+2
default: printf("8mm");
Error: label should be constant
not variable }

⑥ label always in
ascending order
in switch cases
false

⑦ main()
{ int i=1;
switch(i)
{ case 1: printf("hello");
case 2: printf("Hi");
default: break;
} }
O/P ⇒ hello Hi

⑧ main()
{ int a=9;
if (a=5)
printf("It is Important");
else
printf("It is not imp");
} }
O/P ⇒ It is important

⑪ ~~int~~ main()
{ int x=123;
int i = printf("C"++);
for (x=0; x<=i; x++)
{ printf("%d", x);
} }
O/P ⇒ C++
0 1 2 3

⑨ ~~main~~ main()
{ int x=3;
if (x>4)
printf("vector");
else
printf("the missing");
} }
O/P ⇒ the missing

⑩ main()
{ int i, j;
i=j=2,3;
while (--i && j++)
printf("%d %d", i, j);
} }
O/P ⇒ 1, 3

⑫ How many times
the loop iterated?
for (i=0; i=10; i+=2)
printf("Hi");
O/P ⇒ Hi will
print infinite time

13) How many time Vedio print
 main()
 { int x;
 for (x = -1; x <= 10; x++)
 { if (x < 5)
 continue;
 else
 break;
 Pf ("Vedio");
 }
 }

O/P ⇒ 0 times

14) main()
 { int i;
 for (i = 0; i < 5; i++)
 { int j = 10;
 Pf ("%d", i);
 i++;
 }
 }

10 10 10 10 10

15) main()
 { int i = 0;
 char ch = 'A';
 do { Puts (ch);
 while (i++ < 5 || i++ < ch <= 'F');
 }
 }

O/P ⇒ A A A A A B C D E F

16) main()
 { int a = 6, b = 4;
 while (a + b)
 { Pf ("a = %d b = %d", a, b);
 a = a / 2;
 b = b % 3;
 }
 }

O/P ⇒

6	4
3	1
1	1
0	1
0	1
...	...
...	...

17) main()
 { int i, j, x = 0;
 for (i = 0; i < 5; i++)
 { x = i + j + 1;
 Pf ("x = %d", x);
 }
 }

O/P ⇒ 6

```

18) main()
{ int x=3, counter=0;
  while (x>1)
  { ++counter;
    x--;
  }
  printf("%d", counter);
}
OP ⇒ 2

```

20) `int check (static int static, int)`
 ↑
 error
 Can't use static as formal argument

```

21) main()
{ register a, b, x;
  sf("%d %d", &a, &b);
  x = a + b;
}
error, register address
Can't be used

```

24) Every function must return a value
 → false

```

25) main()
{ static char s[25] = "Vector";
  int i=0;
  char ch;
  ch = s[++i]; printf("%c", ch);
  ch = s[i++]; printf("%c", ch);
  ch = i++[s]; printf("%c", ch);
  ch = ++i[s]; printf("%c", ch);
}

```

OP ⇒ e e c u

```

19) int main()
{ int i, a[] = {2, 4, 6, 8, 10};
  change(a, 5);
  for (i=0; i<5; i++)
    printf("%d ", a[i]);
  return 0;
}
void change (int *b, int n)
{ int i;
  for (i=0; i<n; i++)
    *(b+i) = *(b+i) + 5;
}
OP ⇒ 2, 15, 6, 8, 10

```

22) A function assumes int data type as return type, if no return is specified in function prototype.

23) A function type declared as char f ~~actually~~ actually return. char a) ASCII value

```

26) main()
{
    int x;
    x = 3;
    f(x);
    printf("MAIN");
}

f(int n)
{
    printf("f");
    if(n == 0)
        f(n-1);
}

```

FFFF MAIN

```

27) { auto int i = 5;
    static int a = i + 5;
    printf("%d", a);
}

```

error

28) Register variable are treated as static variable if CPU registers are not free.
 → false
 TOP ⇒ auto

29) Scope of a global variable which is declared static is within the file.

```

30) void fn(int, int);
    main()
    {
        int a = 5;
        printf("In main: %d %d", a++, ++a);
        fn(a, a++);
    }

    void fn(int a, int b)
    {
        printf("fn: %d %d", a, b);
    }

```

O/P ⇒ 67
 87

```

31) void main()
{
    f();
    f();
    f();
    static int i = 1;
    printf("%d", i++);
    i = i--, i++, i = i - 2;
    i = (i++, i = i - 2);
}

```

O/P ⇒ 1 -

(32) register help in faster execution.

```
33) int fn(int v)
{
    if (v == 1 || v == 0)
        return 0;
    if (v % 2 == 0)
        return fn(v/2) + 2;
    else
        return fn(v-1) + 3;
}

main()
{
    printf("%d", fn(7));
}
```

O/P = 11

```
34) void main()
{
    int i = 3, *j, **k;
    j = &i;
    k = &j;
    printf("%d %d %d", *j, *k, **k);
}
```

OP $\Rightarrow$ 333

```
36) main()
{
    int i = 300;
    char *ptr = &i;
    *++ptr = 2;
    printf("%d", i);
}
```

```
37) main()
{
    int a, b, c, d;
    char *p = (char *)0;
    int *q = (int *)0;
    float *r = (float *)0;
    double *s = 0;
}
```

$\Rightarrow$ 1 4 4 8

```
38) main()
{
    char *ptr;
    char mystring[] = "abcdefg";
    ptr = mystring;
    ptr += 5;
    printf("%s", ptr);
}
```

O/P $\Rightarrow$ fg

```
39) int cap(int);
main()
{
    int n;
    n = cap(6);
    printf("%d", n);
}

int cap(int n)
{
    if (n <= 1)
        return 1;
    else
        return (cap(n-3) + cap(n-1));
}
```

40) the invalid pointer ~~arithmetic~~ arithmetic
is allowed of them add, mult, div

41) what is the diff b/w the following
~~data is constant~~ Const int *ptr
int *const ptr
Const int *const ptr

42) when the digit 0 is entered from keyboard
what is sent to

48

43) label in switch case should not
be float. → error.

44) main()
{ int ch=1;
switch(ch)
{ case 1: pf("It is working\n");
~~case~~ vector: pf("It's not working\n");
} }
both

45) ~~auto~~ int a=5 ← error
int main()
{
}

auto variable
should not
declare as global

46) main()
{ char str[]="Genuine";
print(str);
print(char *s)
if(*s)
{ pf(s++) } }
pf("%c", *s);

execute
Continuously
segmentation
fault

```

(4) main()
{ int i=9;
  switch(i)
  { printf("hello");
    case 9: printf("hello");
      break;
    default: ("Hachan");
  }
  }
  O/P = abc

```

```

(7) int counter(int i)
{ static int count=0;
  count = count + i;
  return(count);
}

main()
{ int i, j;
  for(i=0; i<5; i++)
  { j = counter(i);
    printf("%d", j);
  }
}
O/P = 15

```

(9) static → scope of global variable declared in —

```

(10) #include <stdio.h>
void main()
{ int i=3, j=1, k;
  j = &i;
  k = &j;
  printf("%d %d %d", i, j, k);
}
O/P = 3 3 3

```

(11) strings

(12) realloc

```

(13) main()
{ char *p = "agam";
  printf("%d", ++*(p++));
}

```

Runtime error

```

(6) void fun(void *p);
int p;
main()
{ void *vptr;
  vptr = &p;
  fun(vptr);
  return;
}

```

```

void fun(void *p)
{ int *a;
  a = (int *) &p;
  printf("%d", **a);
}

```

O/P = 0

```

(8) int fun(char *str)
{ char *ptr = str;
  while(*ptr++)
  { return ptr - str - 1;
  }
}

void main()
{ printf("%d", fun("vechi"));
}
O/P = 0

```

(14) main()

```

{ char thought [2][30]
  = "Don't walk
  in front of me ---"
  "I am not
  follow";
  printf("%c %c",
    *(thought[0][9]),
    *(thought[1][9]));
}

```

O/P = K -

15) main()
int i=257
int *iptr=&i;
printf("%d %d", *((char *)iptr),
*((char *)iptr+1));

O/P ⇒

11

16) main()
static int arr[]={0,1,2,3,4};
int *ptr=&arr[0], *ptr2=&arr[1], *ptr3=&arr[2];
ptr++;
printf("%d %d %d", *ptr, *ptr2, *ptr3);
*++ptr;
printf("%d %d %d", *ptr, *ptr2, *ptr3);

O/P ⇒

11	1
22	2

17) main()
void *vp;
char ch='g', *cp="goofy";
int j=20;
vp=&ch;
printf("%c", *((char *)vp));
vp=&j;
printf("%d", (int *)vp);
vp=cp;
printf("%s", (char *)vp+3);

O/P ⇒

g2oofy

18) main()
int a,b,c,d;
char *p=(char *)0;
int *q=(int *)0;
float *r=(float *)0;
double *s=0;
a=(int)(p+1);
b=(int)(q+1);
c=(int)(r+1);
d=(int)(s+1);

1448

```

19) main()
{ char a[5][5], flag;
  a[0][0] = 'A';
  flag = (a == *a) & & (*a == a[0]);
  printf("%d", flag);
}

```

O/P = 1

```

20) main()
{ union u;
  int i;
  char c[2];
}

```

```

{ union u;
  u.i = 256;
  printf("%d %d %d", u.i, u.c[0], u.c[1]);
}

```

O/P = 513 0 1

21) structure st can't take the structure st itself as its member

TRUE

22) enum (false, true)

```

main()
{ int i = 1;
  do
  { printf("%d", i);
    i++;
    if (i < 15)
      continue;
  }
  while (false);
}

```

O/P = 1

23) enum mode { green, red, orange }
 Green = Green + 1
 error.

24) Nested structure

```

25) main()
{ enum bool { true, false };
  O/P = not true.
}

```

```

26) main()
{ struct xx;
  int x = 3;
  char name[] = "hello";
}

```

27) 0

28) main()

```

{ union
{ struct
{ char c[2];
  char ch[2];
} s;
struct
{ short int i;
  short int j;
} st;
} x = { 12, 1, 15, 1 };
printf("%d %d", u.st.i, u.st.j);
}

```

O/P = 268 271

```

39 #define big(a,b) a>b ? a : b
    #define swap(a,b) temp=a
    a=b
    b=temp

void main()
{
    int a=3, b=5, temp=0;
    if ((3+big(a,b)) > b) swap(a,b);
    printf("%d %d", a, b);
}

```

O/P $\Rightarrow$ 50

30 in macro call the control is passed to the macro $\rightarrow$ false

31 macro have local scope $\rightarrow$ false

32 what is type of variable f2 & p2

```

# define float_ptr float *
# define int_ptr int *
float_ptr f1, f2;
int_ptr p1, p2;

```

$\rightarrow$ float, int

```

33 #define sum(a,b,c) a+b+c
    #define avg(a,b,c) sum(a,b,c)/3

```

```

# define geq(a,b,c) avg(a,b,c)
    >= 60

```

```

# define leq(a,b,c) avg(a,b,c)
    <= 60

```

```

# define des(a,b,c,d)
    (d==1 ? geq(a,b,c) :
    leq(a,b,c))

```

```

main()
{
    int num=70;
    char ch='0';
    float f=2.0;
    if des(num, ch, f, 0) puts("leq");
    else puts("geq");
}

```

O/P $\Rightarrow$ geq

```

34 int main()
{
    FILE *fp;
    char ch, str[7];
    fp=fopen("try.c", "r");
    fseek(fp, 9L, SEEK_CUR);
    fgetc(str, 5, fp);
    puts(str);
    return 0;
}

```

O/P $\Rightarrow$ agpu

35 function call `fseek(fp, 0, 0)` is same as `rewind(fp)`

```

36 main()
{
    FILE *fp;
    printf("%d", ++EOF);
}

```

error

37 text file = vector

```

main()
{
    FILE *fp;
    ptr=fopen("file.txt", "r");
    printf("%c", fgetc(ptr));
    fseek(ptr, 2, 0);
    printf("%c", fgetc(ptr));
    fseek(ptr, -2, 1);
}

```

O/P $\Rightarrow$ vtc

39) preprocesses

40) what is diff b/w the
`#include <file.h>` $\uparrow$ predefined
`#include "file.h"` $\uparrow$ ~~use~~ define.

41) main()

```
int arr[2][3][2] = { {12, 43, 17, 83, 13, 43},  
                      {12, 23, 12, 33, 13, 43} };  
printf("%d", **(*arr+2) + 2 + 7); }
```

O/P = 12

42) what is size of union.

```
union {  
    int x;  
    char y;  
};
```

O/P $\Rightarrow$ size $\Rightarrow$ 8 byte

PAPER - 2

1

main()

{ int a=2, x=10;

if(a==2)

if(x==8)

pf("hello")

pf("Hi")

(Hi)

2

{ int i=1, j=9;

if(i>=5 && j<5)

i=i+2

pf("x.d", i);

(1)

3

{ int x=3

if(x==2);

x=0;

if(x==3)

x++;

else x+=2;

pf("x.d", x);

(2)

4 { int a = 0; }
 if (a < 4 > 12)
 printf("Right")
 else
 printf("Wrong");
 }

Wrong

if (0)
 else
 Wrong

5 main() { int i, j;
 i = j = 2, 3;

i = 2
 j = 3

while (--i && j++)
 printf("%d, %d", i, j);
 return 0;
 }

1, 3

6 The minimum no. of times while loop is executed is?
 → Zero

7 { int x = 0;
 for (x = 1; x < 4; x++)
 printf("x = %d", x);
 }

4

8 main()

{ void *VP;

char ch = 'J', *CP = "JACK";

int j = 65;

VP = &ch;

printf("%d", *(char*)VP); VP = &j;

JACK

```

    Pf("%C", *(int*)VA), ← A
    VP = CP;
    Pf("%S", (char*)VA+2), ← CR
    return 0;
}

```

9 main() { Pf("%C", "abcde"); return 0; }

(a)

10 Array name represents the Base address of the starting element.

11 If size of an array is less than the number of initializer, the size grow
→ false

12 main() { char stop[] = "Never ever say no";
char *chp, c = 'e';
int i, j;
chp = strchr(stop, c);
i = chp - stop;
for (j = 0; j <= i; j++)
Pf("%c", stop[j]);
}

Never ever

(✓)

13 main()
 { int buf[5]
 int *bptr = &buf[3];
 bptr[-1] = 2; bptr[-2] = 1;
 bptr[3] = 0;
 pf("%d", buf[2]);
 }

(2)

14 main()
 { int A[] = {1, 2, 3, 4, 5};
 int x;
 x = *A + 1 - *A + 3;
 pf("%d", x);
 }

(X)

(4)

1 + 1 - 1 + 3 = 4

15 main()
 { char *p = "vector institute";
 *p++;
 pf("%s", p);
 p++;
 pf("%s", p);
 }

(X)

ector institute

ctor institute

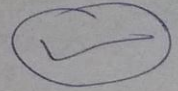
16 ~~auto~~ int a = 5; ← Compile time error
 int main()
 { int x;
 x = a + a + a + a + a;
 pf("%d", x);
 }

(X)

17 main()

```
{ int i, arr = {2, 4, 6, 8, 10};  
  change(a, 5);  
  for (i=0; i<=4; i++)  
    printf("%d", arr[i])  
}
```

```
void change (int *b, int n)  
{  
  int i;  
  for (i=0; i<n; i++)  
    *(b+i) = *(b+i) + 5;  
}
```



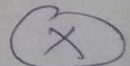
ans
2, 15, 11, 13, 15

18 #include <stdio.h>

```
int *check (static int, static int);  
int main ()
```

```
}
```

Compile time error
we can't
take static in function.



19

```
int a=5;
```

```
main()
```

```
{ int x;
```

```
  x = !a + change();
```

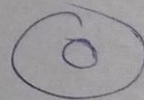
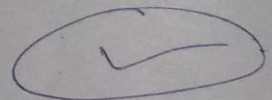
```
} printf("%d", x);
```

```
int change()
```

```
{ a=0;
```

```
  return;
```

```
}
```



integer data as a return

21 ever function must return false

22 func(int i)

```
if (i > 2)
    return 0;
else
    return i;
```

}

main()

```
int i = 3;
i = func(i);
i = func(i);
printf("i", i);
}
```

1

X

5 15 24 21
0 0 1 1 1 1 0

23 main()

```
static char S[25] = "vector";
int i = 0;
char i = 0;
char ch;
```

```
ch = S[i++]; printf("%c", ch);
ch = S[i++]; printf("%c", ch);
ch = S[i++]; printf("%c", ch);
ch = S[i++]; printf("%c", ch);
```

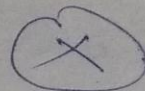
}

X

eeeu

24

```
main()
{
    int x;
    x = 3;
    f(x);
    pf("MAIN");
}
f(int n)
{
    pf("F");
    if (n != 0)
        f(n-1);
}
```



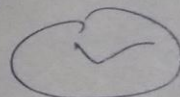
FFFFMAIN

25

what is default storage class of global variable.



extern



26

which storage class may help in faster execution?

→ Register.



27

```
int fn(int V)
```

```
{
    if (V == 1 || V == 0)
        return 1;
```

```
    if (V/2 == 0)
```

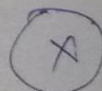
```
        return fn(V/2) + 2;
```

```
    else
```

```
        return fn(V-1) + 3;
```

```
}
```

```
main() { pf("%d", fn(7));
}
```



11

28 main()
 & char thought[2][3] = {"Don't walk in front of
 me - !!", "I am not follow!"}

✓ P f (" \.c \.c", *(thought[0]+9), (* (*thought[0]
 +5)));

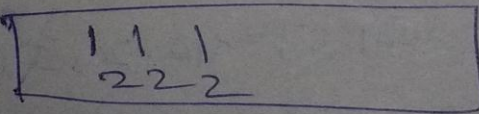
→ 

29 what is the value of a[0][2] in int
 a[3][4] = {1, 2, 3, 4, 8, 15}?

✗ → ○

30 main()
 & static int a[] = {0, 1, 2, 3, 4};
 int *p[] = {a, a+1, a+2, a+3, a+4};
 int *ptr = p;

✗ ptr++;
 p f (" \.d \.d \.d", ptr-p, *ptr-a, **ptr);
 ++ptr;
 p f (" \.d \.d \.d", ptr-p, *ptr-p, *ptr-a,
 , **ptr);

⇒ 

31 main()

& char a[5][5], flag;
 a[0][0] = 'A';
 flag = ((a == *a) && (*a == a[0]));
 P f (" \.d", flag);

o/p ⇒ 1

```

main()
{ char *ptr;
  char mystring[] = "abcdefg";
  ptr = mystring;
  ptr += 5;
  printf("%s", ptr);
}

```

eg

33 mention atleast three differences b/w ~~sizeof()~~ ~~sizeof~~ & ~~strlen()~~ ~~strlen~~

→ not do calculation	→ do calculation
sizeof()	strlen()
→ work on any datatype	→ work for only string
→ it will include '\0'	→ it will not include '\0'
→ size of operator	→ strlen is function.

34 write the prototypes for 1) strcpy() 2) strcat() 3) strlen()

(X) ~~strcpy~~ (char*, char*)
 → ~~strcpy~~ (char*, char*)
 → ~~strcat~~ (char*, char*)
 → ~~int~~ ~~strlen~~ (char*)

35 main()

```

{ int arr[5] = {10, 20, 30, 40, 50};
  printf("%d\n", &arr[4] - &arr[0]);
}

```

4

36 The invalid pointer arithmetic is (are)
→ all are invalid

37 All the string manipulation functions have been included in the string.h header.

38 inter linkage & external linkage

39 parameters of

```
40 int cap(int)  
main  
{ n=cap(6)  
  pf("%d", n);  
  if  
  { int cap(int n)  
    if (n <= 1) return 1;  
    else return (cap(n-3) + cap(n-1));  
  }
```

41 main()

{ int i=20, k=0, j;

for (j=1; j<1; j=1+4*(0/j))

k += j < 10 ? 4 : 3;

pf("%d", k);

}

O/P ⇒ 0

42 main() { char str[] = "Genius";

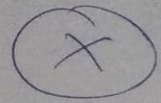
Print(str);

Print(~~str~~ (char *s)

& if (*s) ~~Print~~ (s++);

Printf("%c", *s);

Segmentation
fault



43

How print 50 using P?

main()

{ int a[] = {10, 20, 30, 40, 50};

char *P;

P = (char *)a;

}

Ans → P + 16

P + 16

or
(short *)P + 8

44 #include <stdio.h>

int main()

{ int i;

for (i = 0; i < 5; i++)

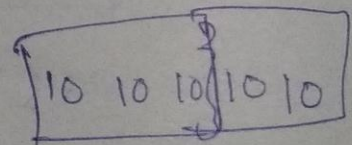
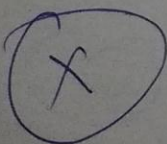
{ int j = 10;

Printf("%d", i);

i++;

return 0;

}



45 main ()

{ int n=10 ;

int fune(int);

printf ("%d", fun(n));

}

int fune(int n)

{ if (n > 0.)

return (n + fune(n-2));

else return 0;

}

Op → 30

30
return (10 + fun(8))

8 + 12 ~~fun(8)~~ return (8 + fun(6))

6 + 6 return (6 + fun(4))

4 + 2 return (4 + fun(2))

2 return (2 + fun(0))